

Project: Lemonade app

About this codelab

1. Before you begin

This codelab introduces a new app called Lemonade that you'll build on your own. This codelab walks you through the steps to complete the project, including setup and testing within Android Studio.

This codelab is different from the others in this course. Unlike previous codelabs, the purpose of this codelab is **not** to provide a step-by-step tutorial on how to build an app. Instead, this codelab is meant to set up a project that you will complete independently, providing you with instructions on how to complete an app and check your work on your own.

Instead of solution code, we provide a test suite as part of the app you'll download. You'll run these tests in Android Studio (we'll show you how to do this later in this codelab) and see if your code passes. This may take a few tries—even professional developers rarely have all their tests pass on the first try! After your code passes all the tests, you can consider this project as complete.

We understand that you might just want the solution to check against. We deliberately do not provide the solution code because we want you to practice what it's like to be a professional developer. This might require you to use different skills that you don't yet have a lot of practice with, such as:

- Googling terms, error messages, and bits of code in the app that you don't recognize;
- Testing code, reading errors, then making changes to the code and testing it again;
- Going back to previous content in Android Basics Unit 1 to refresh what you've learned;
- Comparing code that you know works (i.e. code that is given in the project, or past solution code from other apps in Unit 1) with the code you're writing.

This might seem daunting at first, but we are 100 percent confident that if you were able to complete [Unit 1](#), you are ready for this project. Take your time, and don't give up. You can do this.

Prerequisites

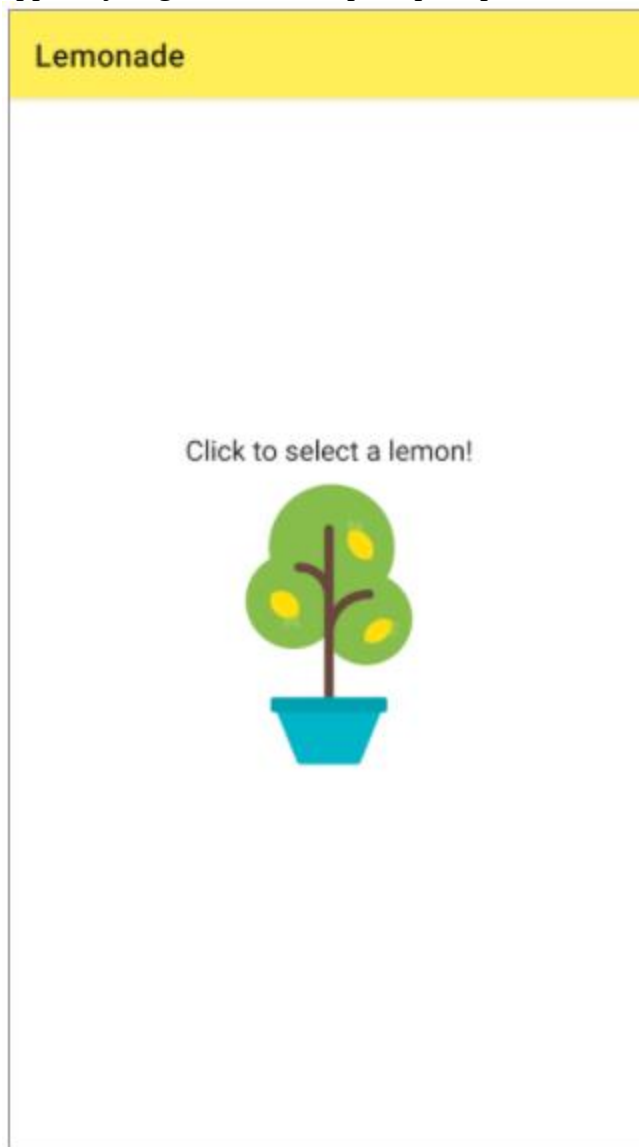
- This project is for users who have completed [Unit 1](#) of the Android Basics in Kotlin course.

What you'll build

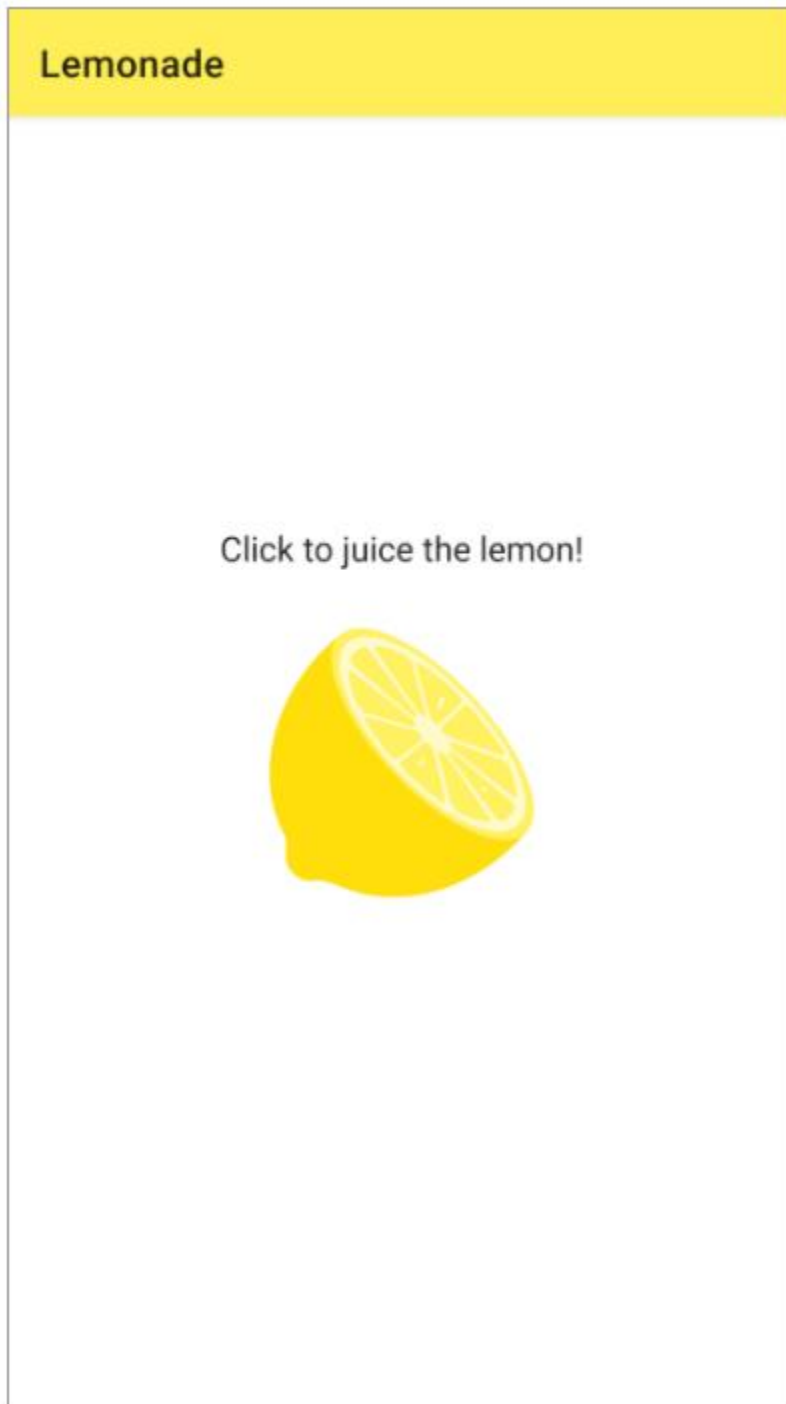
- You'll build a simple Lemonade app using the skills you learned in Unit 1.

What you'll need

- A computer with Android Studio installed.
- [2. App overview](#)
- Welcome to Project: Lemonade app!
- We have recruited you to our team to help us bring our vision of making digital lemonade to life. The goal is to create a simple, interactive mobile app that allows you to juice lemons until you have a glass of lemonade. Consider it a metaphor, or maybe just a fun way to pass the time!
- The finished Lemonade app will consist of a single screen. When users first launch the app, they're greeted with a prompt to pick a lemon by tapping a picture of a lemon tree.



- Tapping the lemon tree presents the user with a lemon that they can tap to "squeeze" for an unspecified number of times (the exact number of required squeezes is randomly generated) before moving to the next screen.



-
- Once the user has tapped to squeeze the lemon the correct number of times, they will see an image of a glass to "drink" the lemonade.

Lemonade

Click to drink your lemonade!



-
- After clicking to drink the lemonade, the glass appears empty, and the user can tap the image again to return to the first screen, and select another lemon from the tree.



-
- The app is built around simplicity, and is organized in a single activity. The different states of the app (whether the user is selecting a lemon, squeezing the lemon, drinking the lemonade, and, finally, the empty glass) are represented by something called a *state machine*. This sounds like a fancy theoretical term, but all this means is that the app's state (i.e. which text and image is shown to the user) is determined by a variable that contains the app's state (`select`, `squeeze`, etc.). The app's state is updated, along with any other needed variables, and then the UI is configured (setting the image and text) separately once all the updates have been made.
- All the variables for the app's state have been defined for you. Your job is to build the app's layout and implement the logic so that the UI transitions between each state as expected.
- **Testing your code**
- For the Lemonade app (and future projects), you'll be provided with some automated tests that you can use to verify that your code works as expected.

- What are automated tests, exactly? In software development, you can think of a "test" as code that verifies that other code is working. This is done by checking outputs (such as the contents of UI elements on the screen) to see if they make sense based on inputs, known as "test cases." The starter project for the Lemonade app includes a few tests that you'll be able to run to make sure you've implemented the logic correctly. We'll discuss the tests in more detail later. For now, it's time to download the starter code and start building the Lemonade app.

3. Get started

Download the project code

Note that the folder name is `android-basics-kotlin-lemonade-app`. Select this folder when you open the project in Android Studio.

Starter Code URL:

<https://github.com/google-developer-training/android-basics-kotlin-lemonade-app>

Branch name with starter code: `main`

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. Check and confirm the branch name matches with the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.

main

4 branches

0 tags

Switch branches/tags

Find or create a branch...

Branches

Tags

✓ main

default

room

starter

test_solution

[View all branches](#)

ns

Update issue templates

Update Gradle and library

Update Gradle and library

Initial commit

Initial commit

Initial commit

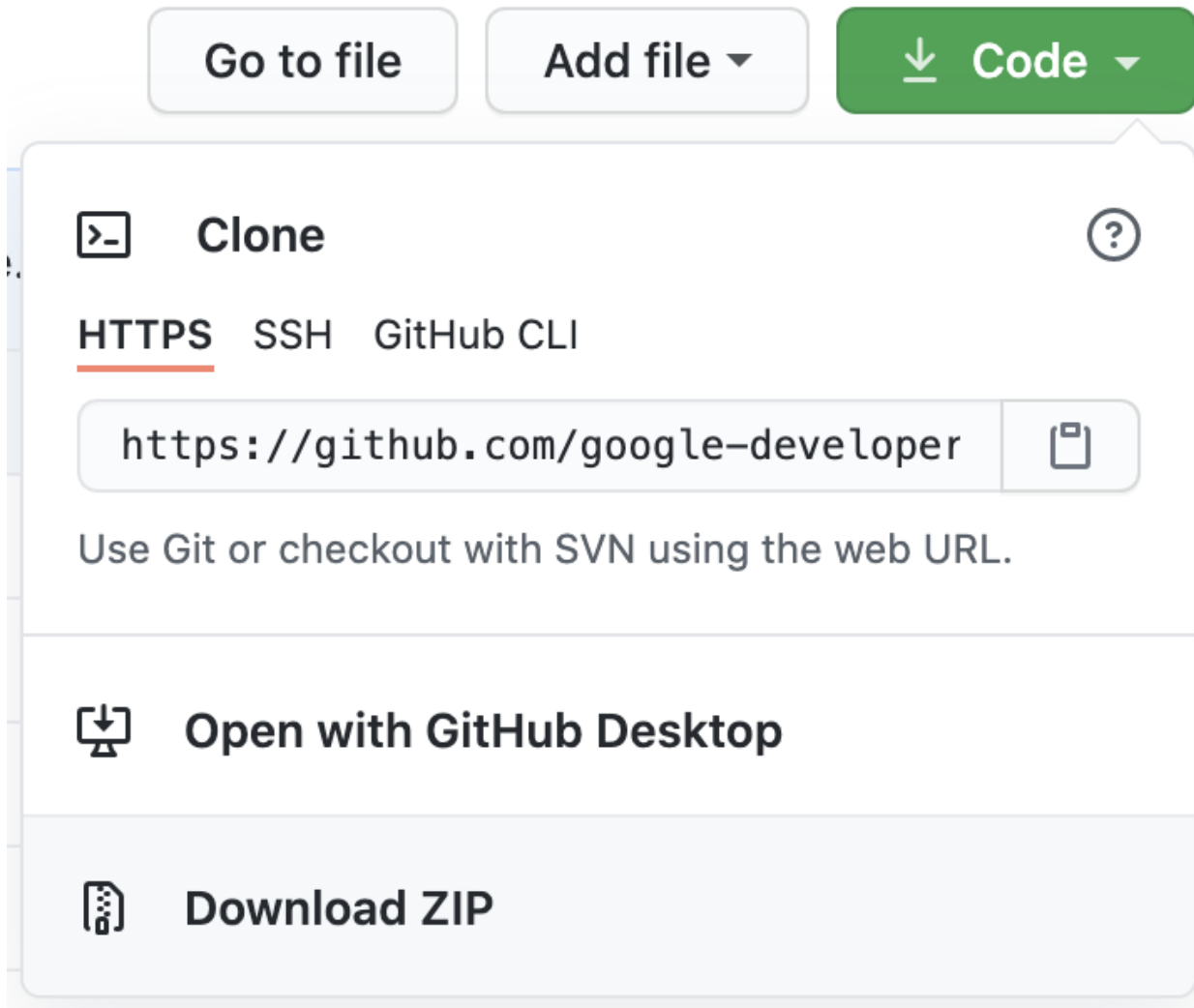
Initial commit

Initial commit

README.md

Initial commit

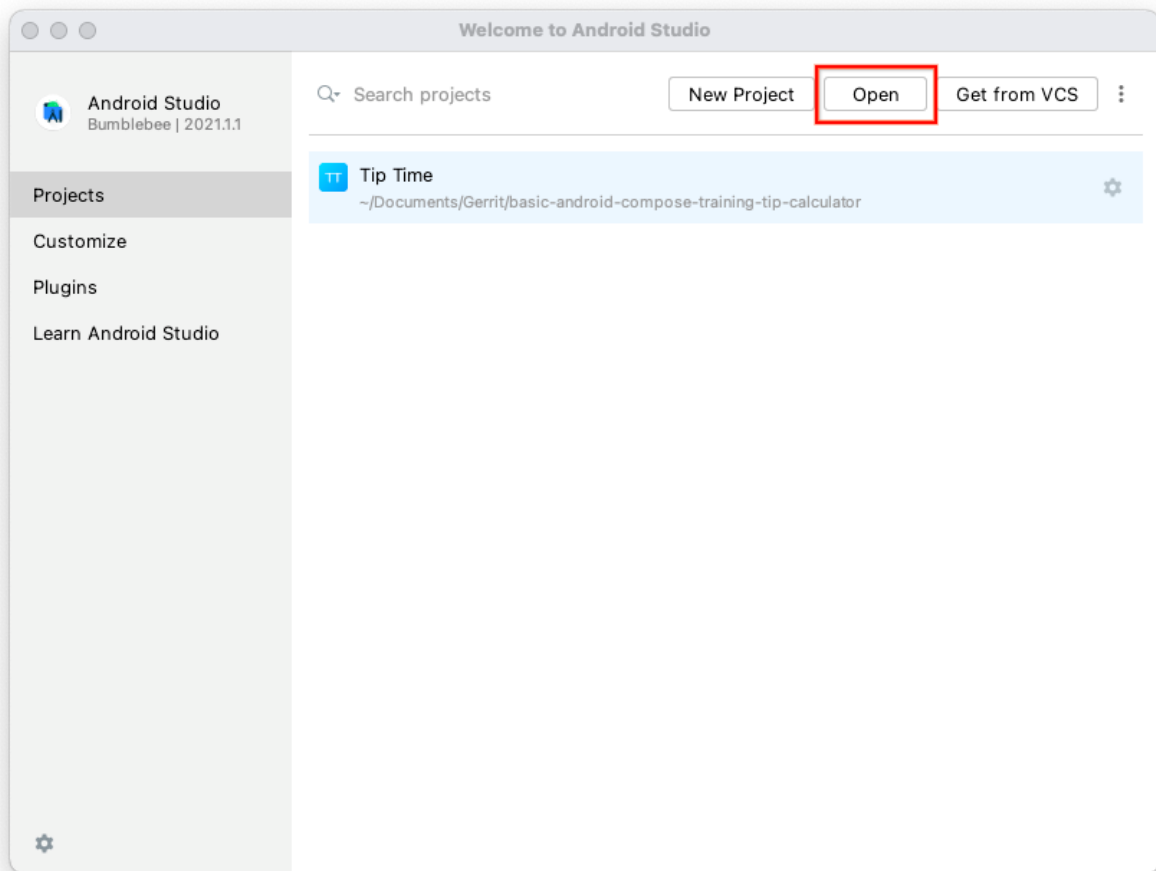
- On the GitHub page for the project, click the **Code** button, which brings up a popup.



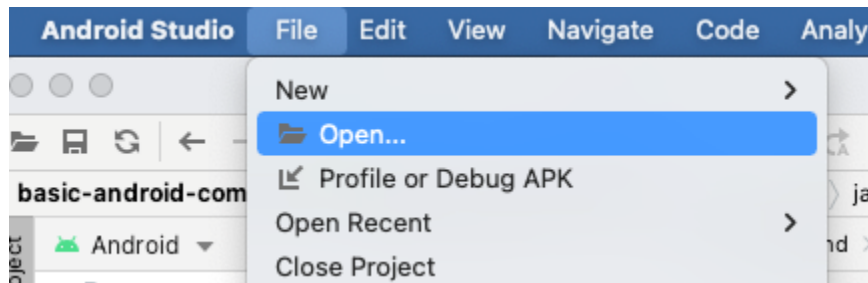
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

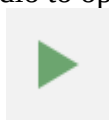
1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.

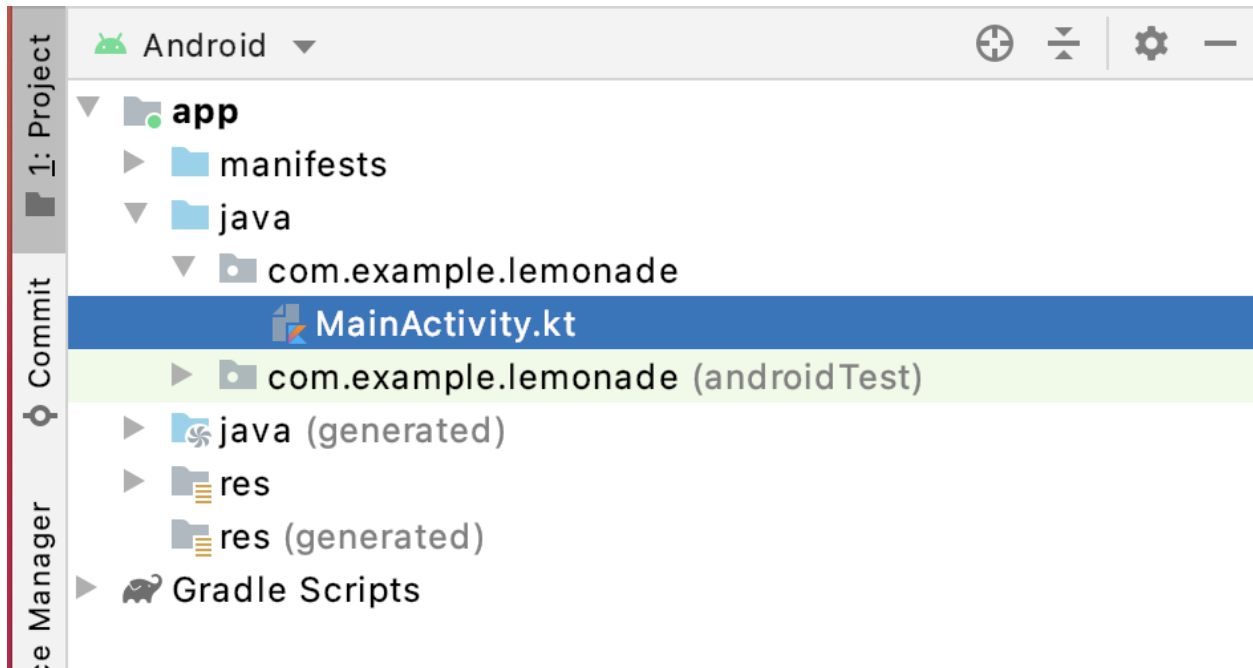


3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



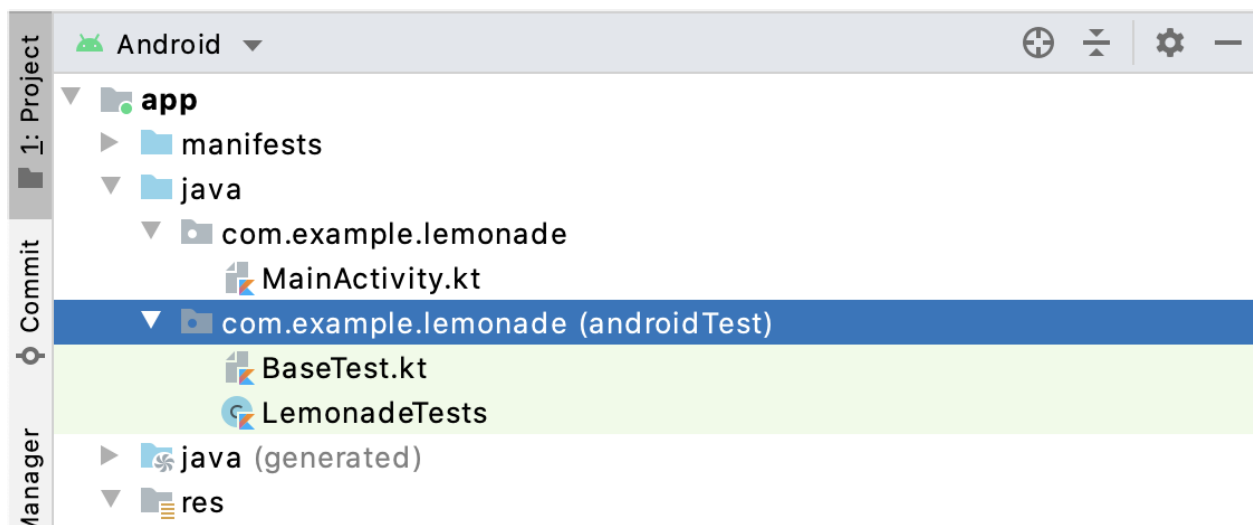
6. Click the **Run** button to build and run the app. Make sure it builds as expected.

Take a moment to familiarize yourself with the starter project. Pay particular attention to the `MainActivity.kt` file.



In the `MainActivity.kt`, you'll find several variables used to represent the app's current state. You'll use these in a later step to make the app interactive. While the amount of code here may seem overwhelming, you won't need to modify any of the code that isn't marked with a `TODO`. Specific instructions are provided on the following pages.

You'll also notice that the project also includes another package, **`com.example.lemonade (androidTest)`**.



This includes the automated tests you'll use to verify the functionality you implement in `MainActivity.kt` is correct. Again, more on that later. For now, you're ready to start building your app, starting with the user interface.

4. Build your user interface

The Lemonade app requires just a basic layout; you only need two views to implement all its functionality.

1. A `TextView` which provides instructions to the user.
2. An `ImageView` which shows a graphic based on the current state of the app (e.g. a lemon to be squeezed).

You'll build this layout in `activity_main.xml`.



Using your knowledge of the layout editor, your goal is to build a layout that looks something like the below, with both views centered onscreen and the `TextView` above the `ImageView`.

Lemonade

Click to select a lemon! I Text size: 18sp



Tip: You've already used a `ConstraintLayout` when working with Android Studio's layout editor. Constraint layouts work well for creating layouts of stacked views such as in the

Lemonade app. If you need a refresher on `ConstraintLayout`, refer back to the [Create a Birthday Card App](#) codelab.

5. Make your app interactive

Once your layout is complete, open up `MainActivity.kt`. This is where you'll implement all the app's logic. You'll notice there's already quite a bit of code. There are also many comments marked `// TODO:` (example below). **These are tasks for you to complete.**

```
113  /**
114   * Set up the view elements according to the state.
115   */
116  private fun setViewElements() {
117      val textAction: TextView = findViewById(R.id.text_action)
118      // TODO: set up a conditional that tracks the lemonadeState
119
120      // TODO: for each state, the textAction TextView should be set to the corresponding string from
121      // the string resources file. The strings are named to match the state
122
123      // TODO: Additionally, for each state, the lemonImage should be set to the corresponding
124      // drawable from the drawable resources. The drawables have the same names as the strings
125      // but remember that they are drawables, not strings.
126  }
```

There are three basic things you'll need to implement to get the lemonade app working.

1. Configure the `lemonImage` `ImageView` to respond to user input.
2. Implement `clickLemonImage()` to update the app's state.
3. Implement `setViewElements()` to update the UI based on the app's current state.

Let's take a look at each task one at a time.

Step 1: Configure the `ImageView`

Tapping the image view should move the app from one state to another. At the end of `onCreate()`, notice that there are two listeners that need to be set.

1. `setOnClickListener()` should update the app's state. The method to do this is `clickLemonImage()`.
2. `setOnLongClickListener()` responds to events where the user long presses on an image (e.g. the user taps on the image and doesn't immediately release their finger). For long press events, a widget, called a `Snackbar`, appears at the bottom of the screen letting the user know how many times they've squeezed the lemon. This is done with the `showSnackbar()` method.



Note: It's important that you DO NOT change the name of any of the existing methods. Otherwise, the tests may be unable to run.

In the next step, you'll implement the logic for changing the app's state.

Step 2: Implement `clickLemonImage()`

After completing the previous step, the `clickLemonImage()` method is now called each time the user taps the image. This method is responsible for moving the app from the current state to the next and updating any variables as needed. There are four possible states: `SELECT`, `SQUEEZE`,

DRINK, and, RESTART; the current state is represented by the `lemonadeState` variable. This method needs to do something different for each state.

1. **SELECT:** Transition to the **SQUEEZE** state, set the `lemonSize` (the number of squeezes needed) by calling the `pick()` method, and setting the `squeezeCount` (the number of times the user has squeezed the lemon) to 0.
2. **SQUEEZE:** Increment the `squeezeCount` by 1 and decrement the `lemonSize` by 1. Remember that a lemon will require a variable number of squeezes before the app can transition its state. Transition to the **DRINK** state only if the new `lemonSize` is equal to 0. Otherwise, the app should remain in the **SQUEEZE** state.
3. **DRINK:** Transition to the **RESTART** state and set the `lemonSize` to -1.
4. **RESTART:** Transition back to the **SELECT** state.

Once you've handled all the updates and transitions between states, be sure to call `setViewElements()` to update the UI based on the new state.

Tip: A `when` statement is a great way to check multiple possible conditions of a single value, such as the `lemonadeState`.

Step 3: Implement `setViewElements()`

The `setViewElements()` method is responsible for updating the UI based on the app's state. The text and image should be updated with the values shown below to match the `lemonadeState`.

SELECT:

- **Text:** Click to select a lemon!
- **Image:** `R.drawable.lemon_tree`

SQUEEZE:

- **Text:** Click to juice the lemon!
- **Image:** `R.drawable.lemon_squeeze`

DRINK:

- **Text:** Click to drink your lemonade!
- **Image:** `R.drawable.lemon_drink`

RESTART:

- **Text:** Click to start again!
- **Image:** `R.drawable.lemon_restart`

Tip: The starter code already has a variable for the `lemonImage` when you set the listeners in `onCreate()`.

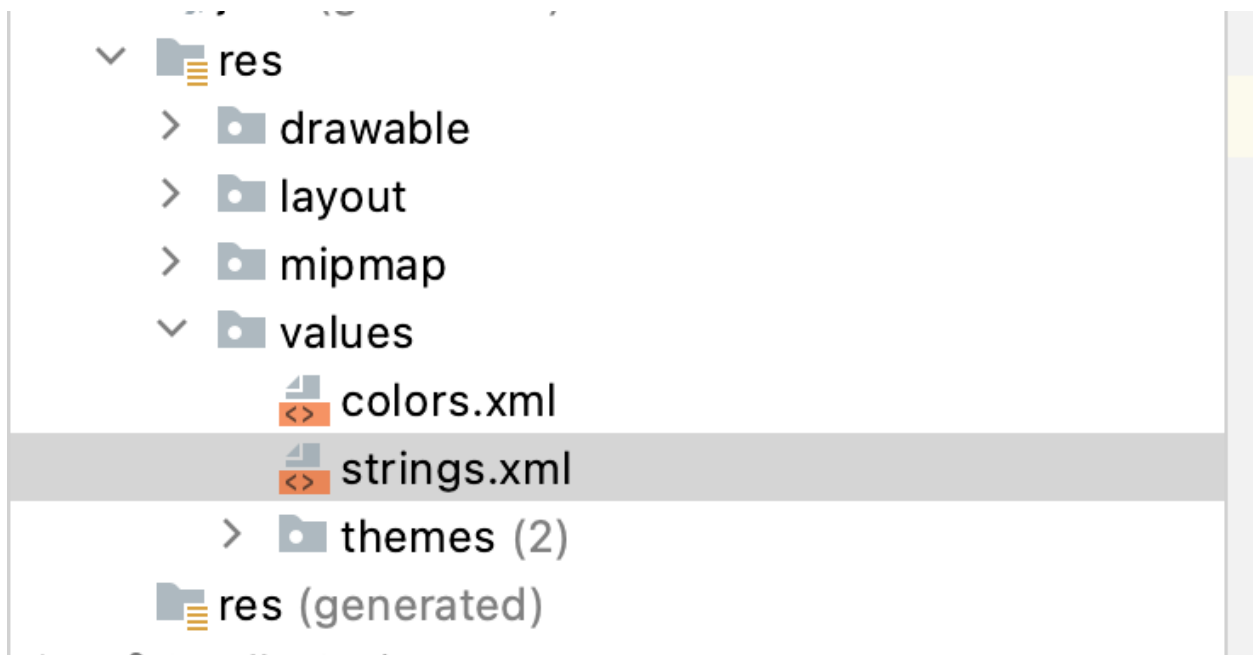
Extra credit: Instead of hardcoding strings in your Kotlin code, consider making these values string resources in `strings.xml`.

How to use string resources

In Android, almost everything is a resource. Defining resources that you can then access in your app is an essential part of Android development.

Resources are used for anything from defining colors, images, layouts, menus, and string values. The benefit of this is that nothing is hardcoded. Everything is defined in these resource files, and then can be referenced within your application's code. The simplest of these resources (and the most common) is using string resources to allow for flexible, localized text.

Strings or static text can be stored in a separate file called `strings.xml` in the `values` subfolder of the `res` folder.



For every piece of text you want to display within your application (i.e the label of a button, or the text inside a `TextView`), you should first define the text in the `res/values/strings.xml` file. Each entry is a key (representing the id of the text) and a value (the text itself). For example, if you want a button to display "**Submit**", add the following string resource to `res/values/strings.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello">Hello!</string>
```

```
<string name="submit_label">Submit</string>
</resources>
```

To access the resource directly in your code, simply use the `getResources.getString()` or `getString()` methods to access the value given the resource id, `R.string.submit_label`:

```
val submitText = getResources().getString(R.string.submit_label)
```

To directly set the text from the string resource to the `TextView` you can call the `setText()` on the `TextView` object, and pass in the resource ID.

```
val infoTextView: TextView = findViewById(R.id.info_textview)
infoTextView.setText(R.string.info_text)
```

String resources can also contain special characters for formatting text. For example, you might have a string resource that allows you to insert another piece of text into the string.

```
<string name="ingredient_tablespoon">%1$d tbsp of %2$s</string>
```

In code, you would access and format the string resource by passing in arguments.

```
getResources().getString(R.string.ingredient_tablespoon, 2, "cocoa powder")
```

When declaring the string resource, each argument is numbered in the order it appears (1, 2, etc.) and has a letter to identify the type (`d` for decimal number, `s` for string, etc.). Arguments of the correct type can be passed into the call to `getString()`.

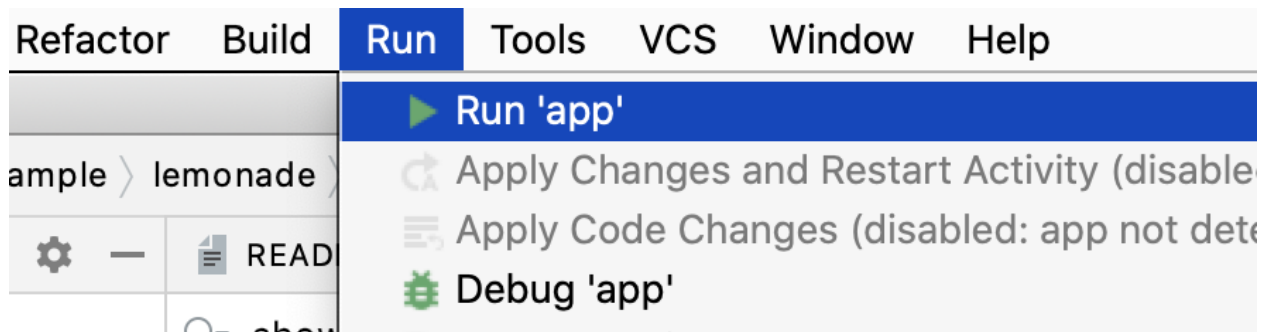
```
2 tbsp of cocoa powder
```

To learn more, refer to the [documentation on string resources](#).

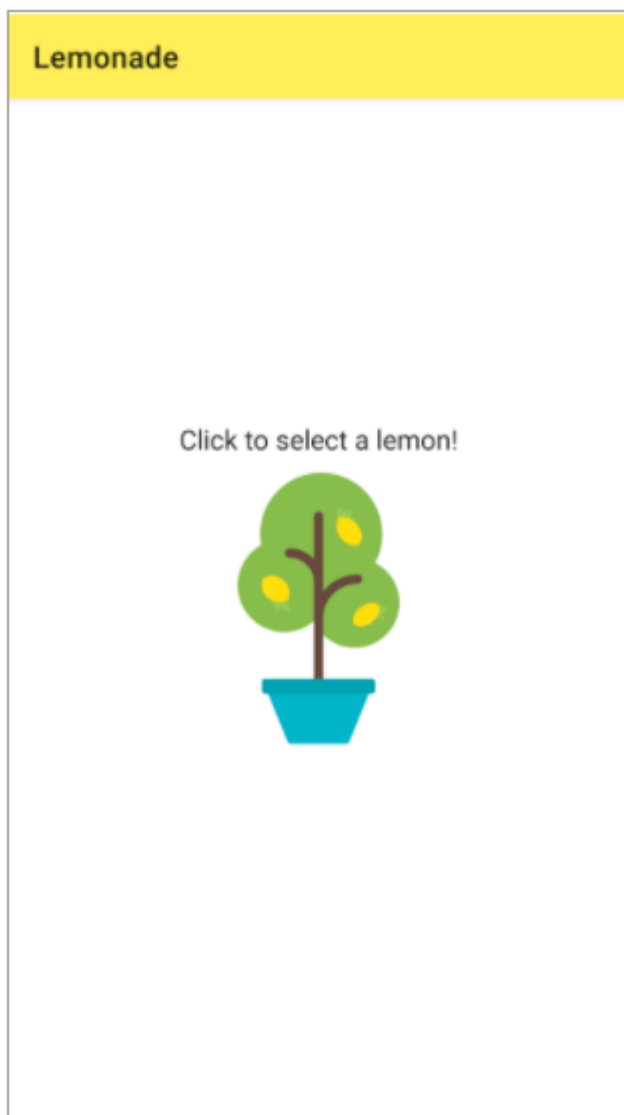
Note: You can see an example of string resources using arguments in the Lemonade app with the `squeeze_count` resource, which is accessed in the `showSnackBar()` method in `MainActivity`.

[6. Run your app](#)

Once you've built the app's UI and implemented the main activity, it's time to see your hard work in action. Run the app using the **Run > Run 'app'** menu and the emulator will launch.



The app should now be fully interactive and you should be able to tap the image view to transition between states.



While the lemon is shown on screen, you can also try long-pressing (press and hold) the `ImageView` to see the `Snackbar` at the bottom revealing the total number of times the lemon has

been squeezed. Take some time to run through the app through all the states a few times. Then, take a moment to congratulate yourself for your hard work!

Tip: If you don't have any virtual devices, you can create one following these instructions in [Create and run your first Android app](#).

7. Testing instructions

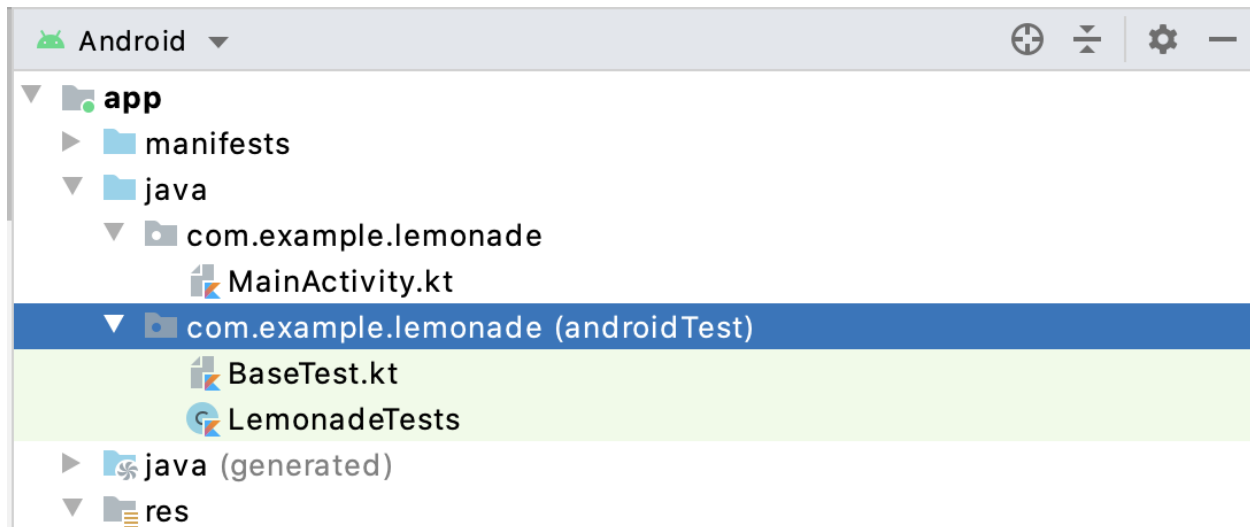
Test your app

You've finished implementing the Lemonade app, but in professional software development, simply writing code is rarely the last step. In addition to application code, professional quality apps also include testing code that is run to verify that the code works as expected and that changes to the code don't introduce any new bugs, a process called automated testing. While teaching automated testing is beyond the scope of this project, the Lemonade app is bundled with some tests to help you verify that you implemented the project correctly. You can use this as a form of self-grading to see if you've passed all the project requirements and if any changes are needed to your app.

What exactly is a "test?" Tests are simply pieces of code included in your Android Studio project that run part of your app's code and can either "pass" or "fail" depending on whether or not your app's code behaves as expected.

Tip: Typically, test failure messages will be specific to the test function. In the example below, the failing test checks the contents of a text view. However, if you're ever unsure of what a test failure message means, you can always paste part of the error into Google. Chances are, someone has asked the same question before!

So where do you find and run your app's tests? The tests for the Lemonade app are found in the testing *target*. A **target** is just a software development term for a collection of classes that are bundled together. For example, the Lemonade app exists in a target called "app" while the tests exist in a target called "LemonadeTests". While the LemonadeTests target can access code from the app target, they're completely separate and the app's code does not contain any of the testing code.



When viewing the files in "Android" view, the test target will appear with the same package name as the app, but with (androidTest) in parentheses.

There are also a few key terms to know when referring to testing code.

- **Test Suite** - the target that includes all your *test cases*.
- **Test Case** - a class consisting of individual tests for related functionality (the Lemonade app had just a single test case but larger apps often have many more).
- **Test** - A function that tests one specific thing.

A test case can have multiple tests, and your project's test suite can have multiple test cases.

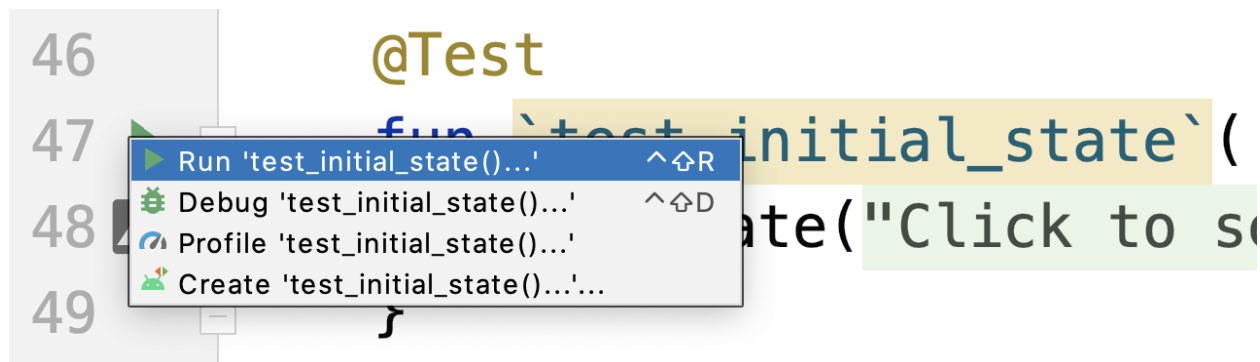
Running your tests

To run your tests, you can do one of the following.

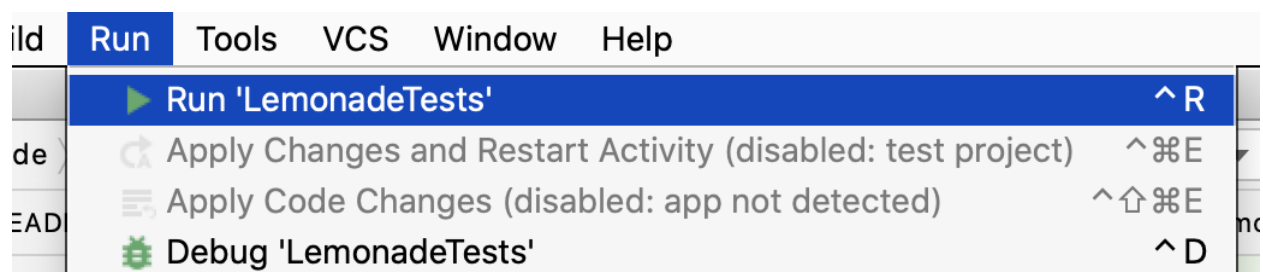
For a single test case, open up a test case class and click the green arrow to the left of the class declaration. You can then select the Run option from the menu. This will run all of the tests in the test case.



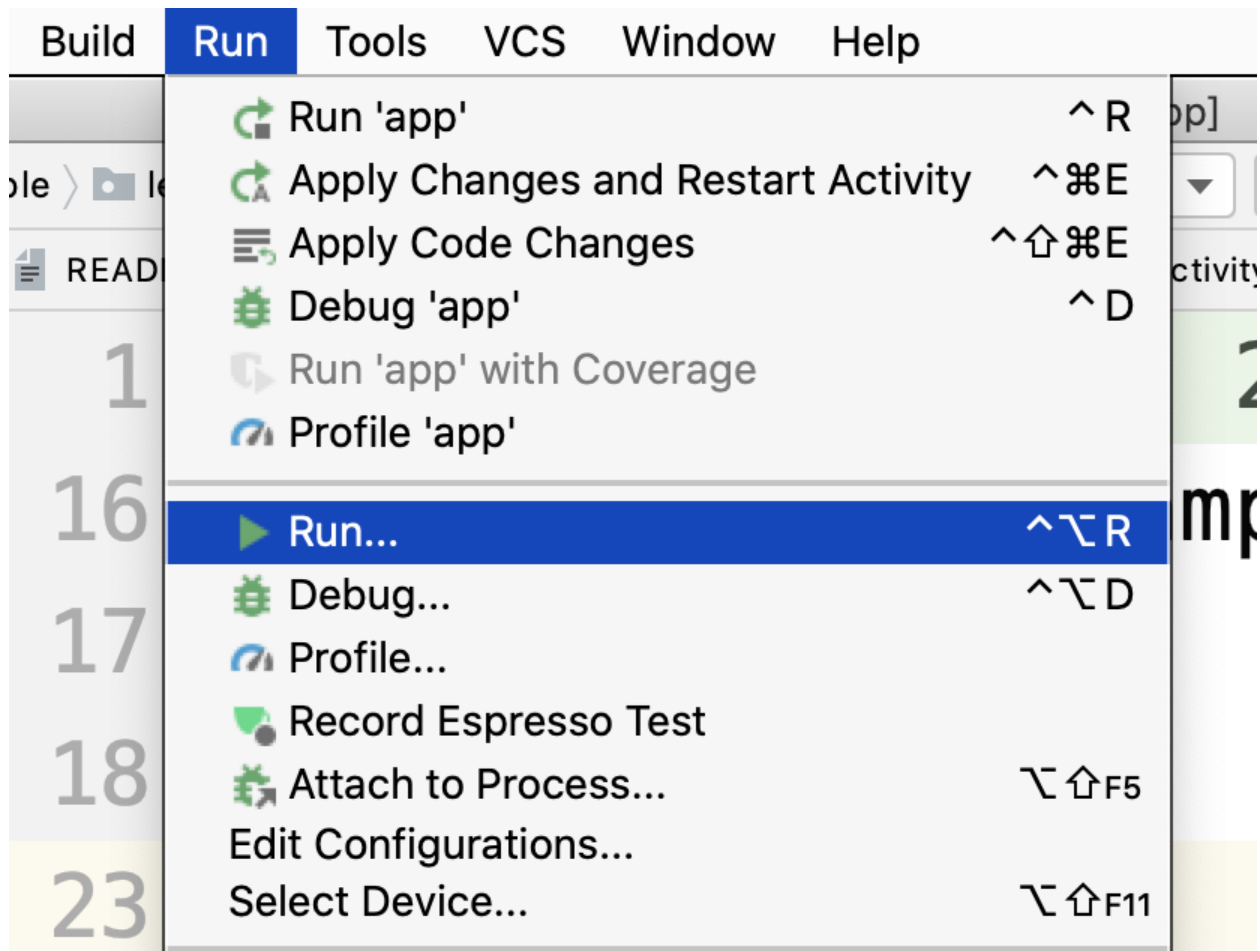
Often you'll only want to run a single test, for example, if there's only one failing test and the other tests pass. You can run a single test just as you would the entire test case. Use the green arrow and select the **Run** option.



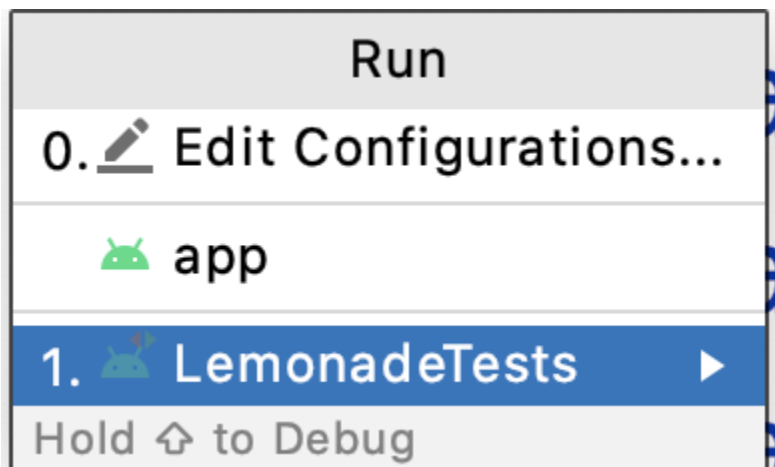
If you have multiple test cases, you can also run the entire test suite. Just like running the app, you can find this option on the **Run** menu.



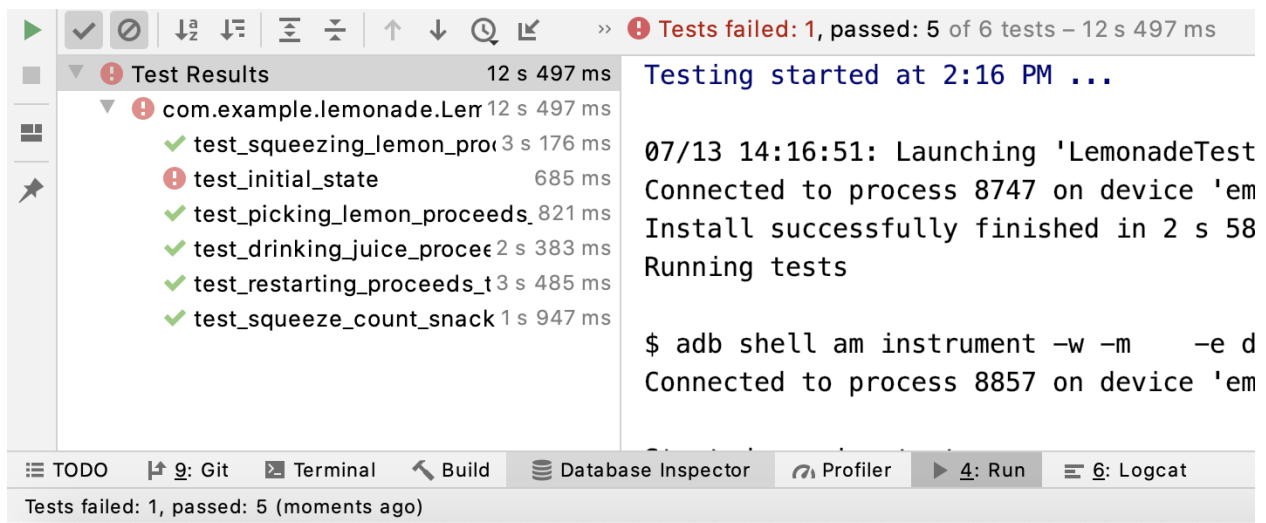
Note that Android Studio will default to the last target that you ran (app, test targets, etc.) so if the menu still says **Run > Run 'app'**, you can run the test target, by selecting **Run > Run**.



Then choose the test target from the popup menu.

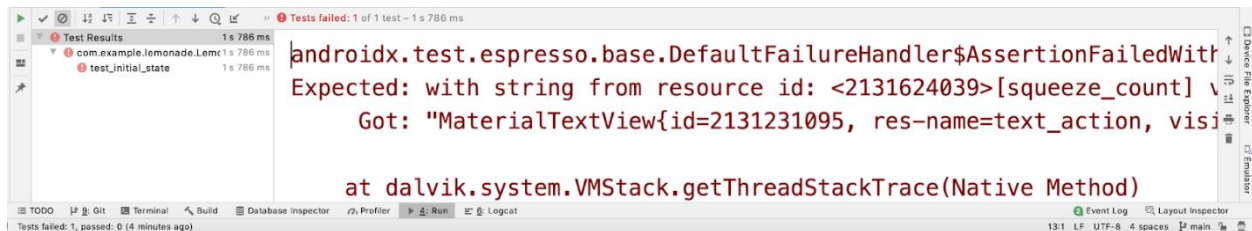


The results of running the tests are shown on the **Run** tab. In the pane on the left, you'll see a list of failing tests, if any. Failing tests are marked with a red exclamation point next to the function name. Passing tests are marked with a green check mark.



Tip: To display passing, failing, or both passing and failing tests in the left pane, you can use the two buttons at the top left. Selecting the checkmark will show passing tests, selecting the icon of a circle with a line through it will list the failing tests. By default, only the failing tests are listed.

If a test fails, the text output provides information to help you fix the problem that caused the test to fail.



For example, in the above error message, the test is checking if a `TextView` is using a specific string resource. However, the test is failing. The text after "Expected" and "Got" do not match, meaning the value the test expected does not match the value from the running app. In this example, the string used in the `TextView` is not in-fact `squeeze_count`, as the test expects.

8. Optional: Share your app!

After you've finished enjoying many glasses of Lemonade, take a screenshot of your favorite screen, and share it on Twitter to showcase what you learned. Tag [@AndroidDev](#) and add the hashtag **#AndroidBasics**.